Taller de Sistemas Embebidos – 2026 2C – Parcial

Apellidos: Delgado Fernandez **Nombres:** Alex Braian

Aciertos (suman): 37 **Yerros** (restan): 7 **Nota:** 4 (cuatro)

Marcar sólo **una de cuatro** opciones

1. En la clasificación general de los Sistemas Digitales presentada en la asignatura, ¿cuál de las siguientes plataformas pertenece específicamente a la categoría de Sistemas Embebidos?
   a. Microcontroladores (uC) - System on a Chip (SoC) y Procesadores Digitales de Señal (DSP).
   b. Circuitos Integrados de Aplicación Específica (ASIC) y VLSI.
   c. Sistemas de Propósito General basados habitualmente en microprocesadores (uP).
   d. Arreglos de Compuertas Programables en Campo (FPGA) y CPLD.

2. En la arquitectura de software de tipo Foreground/Background (Super-loop), ¿cómo se reparten el trabajo el primer y el segundo plano?
   a. El primer plano gestiona eventos asincrónicos mediante Rutinas de Servicio de Interrupción (ISR), mientras el segundo plano ejecuta un bucle infinito con las tareas secuenciales.
   b. El primer plano se encarga de la inicialización de registros de memoria, mientras el segundo plano atiende exclusivamente las interrupciones de hardware.
   c. Ambos planos ejecutan tareas periódicas con idéntica prioridad coordinadas por un planificador por turnos (round-robin).
   d. El primer plano realiza las tareas de larga duración mediante consultas por encuesta (polling), mientras el segundo plano administra el kernel del RTOS.

3. En el contexto de desarrollo Bare Metal para sistemas embebidos, ¿qué implica que la aplicación opere directamente sobre el microcontrolador?
   a. Que las tareas se organizan obligatoriamente mediante un esquema de desapropiación guiado por la unidad de gestión de memoria (MMU).
   b. Que el microcontrolador prescinde por completo del uso de reloj del sistema e interrupciones externas.
   c. Que la aplicación se escribe interactuando directamente con los registros de hardware sin interposición de un Sistema Operativo.
   d. Que el código se ejecuta únicamente dentro de un entorno virtualizado en la memoria Flash externa.

4. ¿Cuál es la principal objeción al denominado "código Spaghetti" en el desarrollo de software para sistemas embebidos de complejidad media o alta?
   a. Carece de estructura y dificulta la lectura, lo que ocasiona pérdidas de tiempo y recursos para detectar y depurar errores.
   b. Requiere mayor espacio en la sección de código ejecutable (.text) que la programación estructurada.
   c. Impide que el microcontrolador ejecute rutinas de servicio de interrupción (ISR).
   d. Obliga al compilador a traducir todo el código directamente a instrucciones en ensamblador sin pasar por la fase de preprocesamiento.

5. En el desarrollo de sistemas embebidos, ¿cuál es la principal diferencia funcional entre las herramientas informáticas categorizadas como CASE y aquellas clasificadas como CAE/CAD/CAM?
   a. CASE se utiliza exclusivamente para la fabricación de circuitos impresos, mientras que CAM compila el código escrito en lenguaje C.
   b. CASE reemplaza la necesidad de compiladores en la depuración, mientras que CAD diseña los diagramas de flujo del programa principal.
   c. CASE administra únicamente la adquisición de instrumental de laboratorio, mientras que CAE diseña los programas embebidos.
   d. CASE se enfoca en la ingeniería, análisis y gestión del ciclo de vida del software, mientras que CAE/CAD/CAM se centran en el diseño, simulación y manufactura del hardware.

6. De acuerdo con el modelo de comunicación de drivers de salida por interrupción, ¿cuál es el evento que desencadena la habilitación de las interrupciones en el driver de salida?
   a. La finalización completa de todas las tareas del programa principal en el Super-Loop.
   b. La condición de '¡Cola No Vacía!', indicando que existe al menos un dato listo para ser transmitido.

+4 -1

c. La detección de un nivel alto constante en los pines de entrada del microcontrolador.
d. La deshabilitación definitiva del reloj del temporizador principal (Timer Ticks).

7. En la arquitectura estandarizada de software para microcontroladores ARM Cortex, ¿cuál es el propósito de la capa Hardware Abstraction Layer (HAL)?
   a. Compilar diagramas de estado directamente en lenguaje ensamblador sin pasar por el compilador C.
   b. Gestionar exclusivamente la sincronización de repositorios remotos en servidores como GitHub.
   c. Diseñar los trazados físicos de las pistas de cobre en la placa de circuito impreso.
   d. Ofrecer un conjunto de funciones e interfaces estandarizadas que aíslan al programa de la interacción directa con los registros del hardware.

8. En un esquema de trabajo colaborativo en GitHub, si se desea aportar modificaciones a un proyecto público sobre el cual no se poseen permisos directos de escritura, ¿cuál es la secuencia correcta de operaciones?
   a. Ejecutar un comando Rebase en la consola local directamente apuntado a la IP del servidor sin generar commits.
   b. Hacer un Fork del repositorio, clonar la copia propia localmente, realizar las modificaciones, enviarlas con Push a la cuenta personal y emitir un Pull Request al proyecto original.
   c. Descargar el proyecto como un archivo comprimido ZIP, hacer las modificaciones y enviarlo por correo al autor del repositorio.
   d. Clonar directamente el repositorio original, modificar los archivos en local y hacer un Push directo a la rama principal del propietario.

9. ¿Cuál es la función específica del bus de direcciones dentro de una arquitectura general de computadoras?
   a. Sincronizar la ejecución de las instrucciones internas de la ALU mediante pulsos periódicos de tensión.
   b. Transportar los datos de lectura y escritura entre el procesador y el periférico seleccionado.
   c. Especificar la unidad requerida y la posición exacta de memoria o registro a la que se desea acceder.
   d. Determinar si la transferencia actual corresponde a una lectura o escritura y definir su temporización.

10. ¿Cuál es la razón principal por la que la arquitectura de procesadores Arm Cortex-M está ampliamente difundida entre diversos fabricantes de microcontroladores?
   a. Arm fabrica físicamente todos los chips del mercado y asigna la distribución comercial a empresas secundarias.
   b. Obliga a que todos los microcontroladores utilicen una única distribución rígida de memoria e interfaces idénticas.
   c. Permite únicamente la ejecución de programas desarrollados en lenguaje ensamblador propietario de cada fabricante.
   d. Arm comercializa su núcleo como propiedad intelectual (IP), permitiendo a cada fabricante integrar periféricos y memorias propias.

11. ¿Cuál es la función clave de la unidad de control y decodificación de instrucciones dentro de la CPU?
   a. Interpretar el código de la instrucción capturada para coordinar las secuencias internas, seleccionar registros y comandar la ALU.
   b. Supervisar que los niveles de tensión de la fuente de alimentación no presenten rizado.
   c. Incrementar de forma fija la dirección de memoria tras cada ciclo de reloj sin importar el programa.
   d. Efectuar los cálculos aritméticos de multiplicación y división en punto flotante.Compilar diagramas de estado directamente en lenguaje ensamblador sin pasar por el compilador C.

12. ¿Cuál es la diferencia conceptual entre un microprocesador y un microcontrolador?

a. El microcontrolador integra en un único chip de silicio el procesador, memorias e interfaces programables, mientras que el microprocesador requiere estos elementos en componentes externos.
b. El microcontrolador carece de unidad aritmético-lógica y depende de una CPU externa para realizar operaciones matemáticas.
c. El microprocesador únicamente se utiliza en aplicaciones de tiempo real estricto dentro de vehículos de transporte.
d. El microprocesador contiene unidades analógicas de potencia, mientras que el microcontrolador solo procesa datos digitales de 4 bits.

13. ¿Cuál es el motivo estratégico principal por el cual la empresa Apart Car decidió impulsar el desarrollo de productos propios en lugar de continuar comprando equipos terminados a proveedores externos?
a. Eliminar por completo el uso de componentes electrónicos importados en la fabricación de sus terminales de acceso.
b. Lograr una integración vertical que permita reducir la estructura de compras, adquirir capacidades tecnológicas propias y adaptar las soluciones a las necesidades locales.
c. Reemplazar inmediatamente toda la red de 10 playas de estacionamiento por sistemas totalmente automatizados sin personal en planta.
d. Cumplir con una regulación gubernamental obligatoria que exige el uso de software de código abierto en la Ciudad de Buenos Aires.

14. En la arquitectura modular de la máquina dispensadora de tickets de entrada (PTDM), ¿cómo se estructuran y coordinan las tareas del sistema?
a. Se organizan en la estructura escrutar-procesar-actuar (Sensores, Sistema, Actuadores) y se comunican mediante mensajes.
b. Se emplean interrupciones bloqueantes que detienen la CPU hasta que la barrera confirme mecánicamente su apertura.
c. Cada módulo ejecuta un sistema operativo independiente comunicándose mediante protocolos de red en la nube.
d. Se ejecutan en un bloque monolítico donde los sensores modifican directamente el estado de los actuadores sin mediación lógica.

15. ¿Por qué se considera inaceptable el uso de código bloqueante en el firmware diseñado para este sistema embebido?
a. Porque invalida el uso de mensajes entre los módulos del sistema.
b. Porque incrementa de manera irreversible el consumo de memoria FLASH del microcontrolador.
c. Porque imposibilita la lectura de estados en salidas digitales conectadas a LEDs.
d. Porque impide el comportamiento comunitario de la CPU, evitando que otros módulos se ejecuten de manera cíclica y fluida.

16. Una vez emitido el ticket y levantada la barrera, ¿qué evento le indica al sistema que debe enviar el comando de cierre de barrera ("Close barrier")?
a. El agotamiento de un temporizador fijo e incondicional de 30 segundos.
b. La lectura remota del ticket mediante el escáner de salida.
c. Una señal manual enviada de manera ininterrumpida por el operador desde el servidor.
d. La detección del avance y salida del vehículo mediante la señal "Car leaves" del sensor correspondiente.

17. ¿Cuál es la diferencia fundamental en la generación de salidas entre una máquina de Moore y una máquina de Mealy?
a. En una máquina de Moore las salidas solo se ejecutan al recibir un evento temporal, mientras que en Mealy son continuas.
b. En una máquina de Mealy las salidas se determinan por el tiempo transcurrido, mientras que en Moore dependen de variables internas.

+1

c. En una máquina de Moore las salidas dependen exclusivamente del estado actual, mientras que en una de Mealy se producen en las transiciones.
d. En una máquina de Mealy las salidas requieren estados compuestos, mientras que Moore solo utiliza estados simples.

---

18. De acuerdo con la definición matemática acuñada por David Harel, ¿cuáles son los elementos clave que integran la fórmula de un *statechart*?

a. Clases orientadas a objetos, herencia múltiple e interfaces agregadas a las máquinas de Mealy.
b. Variables globales, punteros a función y estructuras de control `switch` agregados a las máquinas de Moore.
c. Profundidad, ortogonalidad y comunicación por *broadcast* agregados a los diagramas de estados.
d. Temporizadores de hardware, interrupciones y memoria no volátil agregados al autómata de estados.

---

19. ¿Cuál es la función principal de una "guarda" (*guard*) en una transición dentro de un diagrama de estados?

a. Condicionar el paso de la transición a que se evalúe como verdadera una expresión lógica dada.
b. Proteger las variables internas para que no sean modificadas por eventos no autorizados.
c. Asignar valores de salida a las variables globales durante la activación de un estado.
d. Forzar la transición inmediata al estado inicial en caso de error del sistema.

---

20. Al implementar un diagrama de estados en lenguaje C, ¿cuál de las siguientes opciones representa una alternativa estructurada para organizar los cambios de estado y eventos?

a. Convertir las transiciones en rutinas de interrupción no anidadas.
b. Utilizar tablas de estados de una o dos dimensiones.
c. Compilar directamente el diagrama visual en registros de hardware sin pasar por el compilador de C.
d. Evitar las variables de control de estado mediante el uso exclusivo de bucles infinitos `while(1)` desvinculados.

---

21. ¿Qué implica conceptualmente el término "periféricos mapeados en memoria" en la arquitectura de los microcontroladores STM32?

a. Los registros periféricos residen físicamente dentro del bloque principal de la memoria SRAM.
b. Las operaciones de lectura y escritura en los registros de periféricos se ejecutan utilizando las mismas instrucciones y rango de direcciones de 32 bits que el acceso a la memoria.
c. Toda la memoria Flash se reconfigura dinámicamente como registros periféricos durante la ejecución del programa.
d. Los periféricos se gestionan exclusivamente mediante instrucciones de entrada y salida especializadas fuera del mapa principal de memoria.

---

22. En la estructura interna del controlador de entrada de un pin GPIO de STM32, ¿cuál de las siguientes rutas de entrada omite (bypasses) el disparador Schmitt (TTL Schmitt trigger)?

a. La entrada de función alternativa digital (Alternate function input).
b. La ruta de lectura del registro de datos de entrada (Input data register).
c. La ruta de control de escritura del registro de salida (Output data register).
d. La ruta para señales analógicas (Analog).

---

23. Al producirse un reinicio por encendido (power-on reset), los registros de modo `GPIOx_MODER` se cargan con valores iniciales por defecto. ¿Cuál es el valor predeterminado para el registro `GPIOA_MODER`?

a. `0x0000 0000`
b. `0xFFFF FFFF`
c. `0x0000 0280`
d. `0x0C00 0000`

+2 -2

24. ¿Por qué las arquitecturas de microcontroladores integrados como el STM32 no suelen requerir una Unidad de Gestión de Memoria (MMU) para realizar paginación extendida como las computadoras personales?

a. Todos los programas se ejecutan desde una tarjeta SD externa que incluye su propio controlador de direccionamiento.
b. El núcleo ARM Cortex-M4 procesa instrucciones nativas de 64 bits para el cálculo de direcciones.
c. La MPU reemplaza técnicamente la función de traducción de direcciones de la MMU.
d. La memoria física instalada en aplicaciones embebidas típicas es reducida y cabe holgadamente dentro del mapa de direcciones nativo de 32 bits.

25. En la arquitectura ARM Cortex-M, ¿cuál es la diferencia terminológica clave respecto al uso común del término "excepción" en otros lenguajes de programación?

a. En ARM, las excepciones son exclusivas del código de software y no tienen relación con el controlador NVIC.
b. En ARM, el término excepción se aplica unificadamente tanto a las interrupciones iniciadas por hardware como por software.
c. En ARM, se llama excepción solamente a los eventos generados por periféricos de comunicación serie y temporizadores.
d. En ARM, las excepciones son únicamente errores fatales de hardware que detienen el microcontrolador de forma permanente.

26. En el contexto del procesamiento de interrupciones, ¿qué diferencia existe entre los conceptos de "armar" un dispositivo y "habilitar" las interrupciones?

a. Armar es una operación realizada únicamente por hardware automático, mientras que habilitar solo se puede ejecutar en lenguaje ensamblador.
b. Armar implica activar el bit de activación específico del periférico, mientras que habilitar permite que el proceso global de interrupción sea reconocido.
c. Armar borra las banderas pendientes de la pila, mientras que habilitar asigna la prioridad en el registro BASEPRI.
d. Armar se refiere únicamente a los temporizadores internos, mientras que habilitar se aplica exclusivamente a las líneas GPIO externas.

27. Durante el proceso de cambio de contexto invocado automáticamente por el NVIC al reconocer una interrupción, ¿qué ocurre con los registros del procesador si la unidad de punto flotante (FPU) está activa?

a. Se almacena en la pila el contenido de 12 registros básicos más 18 palabras adicionales debidas a la FPU.
b. Todos los registros se transfieren directamente al controlador EXTI para liberar la memoria de la pila.
c. Se guardan únicamente 8 registros del núcleo y se descarta el estado de la FPU para reducir la latencia.
d. El registro PC se borra a cero y la FPU detiene de forma inmediata la ejecución del procesador.

28. Al diseñar una Rutina de Servicio de Interrupción (ISR) bajo buenas prácticas de ingeniería de software para sistemas embebidos, ¿cuál de las siguientes reglas es fundamental cumplir?

a. Incluir bucles de retardo prolongados para asegurar la eliminación del rebote de botones e interruptores.
b. Deshabilitar de forma permanente las interrupciones globales justo antes de retornar mediante el comando de salida.
c. Evitar incluir bucles de espera (delays) y almacenar datos compartidos en ubicaciones de memoria global.
d. Declarar todas las variables generadas únicamente con alcance local para guardarlas en la pila de la función.

29. ¿Por qué el uso de un temporizador por hardware integrado con interrupciones es sustancialmente superior a ejecutar un bucle de retardo por software en el código principal del microcontrolador?

+1

a. Elimina la necesidad de utilizar preescaladores o divisores de frecuencia en el sistema.
b. Incrementa la capacidad total de almacenamiento de la memoria RAM disponible para el programa.
c. Libera al núcleo del microcontrolador del procesamiento constante de temporización, permitiéndole ejecutar otras tareas eficientemente.
d. Incrementa automáticamente la frecuencia de trabajo del reloj maestro interno del microcontrolador.

30. Se desea configurar un temporizador STM32 para generar un evento de actualización exactamente cada 1,0 segundo. Si el reloj de alta velocidad es de 64 MHz y el preescalador se fija en 63999, ¿qué valor debe asignarse al parámetro de Período (`Period`)?

a. 63999
b. 499
c. 1000
d. 999

31. Al migrar la configuración de un temporizador de modo sondeado a modo por interrupción en STM32CubeMX, ¿qué acción es imprescindible realizar?

a. Cambiar el parámetro `CounterMode` de `TIM_COUNTERMODE_UP` a `TIM_COUNTERMODE_DOWN`.
b. Asignar el valor `0x0` al parámetro de Período para forzar un disparo inmediato.
c. Habilitar la casilla de verificación de la interrupción global del temporizador en la pestaña de configuración NVIC.
d. Desactivar la fuente de reloj interno en la vista de asignación de pines (*Pinout*).

32. Un temporizador con contador de 16 bits está configurado en modo ascendente (`TIM_COUNTERMODE_UP`). ¿Qué valor alcanza el contador inmediatamente antes de desbordarse?

a. 4294967295
b. 65536
c. 32767
d. 65535

33. ¿Cuántas líneas de entrada analógica externas e independientes puede aceptar el único ADC presente en la MCU STM32F302R8 de la placa de proyecto?

a. 8 líneas
b. 16 líneas
c. 15 líneas
d. 12 líneas

34. Al iniciar una conversión en un SAR ADC de 12 bits, ¿a qué valor se preestablece el registro de N bits (*N-BIT REGISTER*)?

a. Al valor máximo de escala (111111111111_2)
b. A un valor de medio de escala (1000 en 4 bits o 100000000000_2 en 12 bits)
c. Al valor obtenido en la última conversión realizada
d. A cero en todos sus bits (000000000000_2)

35. Al configurar el parámetro `ClockPrescaler` en la estructura `ADC_InitTypeDef`, ¿qué consecuencia tiene seleccionar un factor divisor mayor (por ejemplo, `ADC_CLOCK_SYNC_PCLK_DIV8` en lugar de `DIV1`)?

a. Reduce la velocidad del reloj del ADC (ADCCLK), disminuyendo la frecuencia de muestreo máxima alcanzable
b. Modifica la alineación de datos del registro N-BIT hacia la izquierda
c. Incrementa automáticamente la resolución del ADC de 6 bits a 12 bits
d. Habilita de forma obligatoria el uso de transferencias DMA continuas

36. Cuando se habilita el envío continuo de solicitudes DMA (`DMAContinuousRequests = ENABLE`), ¿por qué motivo debe configurarse el periférico DMA en modo circular?

a. Para alternar automáticamente la alineación de datos entre izquierda y derecha

+1 -1

b. Para permitir que la lógica SAR aumente la velocidad del reloj ADCCLK
c. Para evitar que se produzca una saturación o desbordamiento al alcanzar el puntero máximo del búfer de memoria
d. Para desactivar el uso del circuito de seguimiento y retención (*Track/Hold*)

37. ¿Cuál de las siguientes aplicaciones de una señal PWM aprovecha directamente la variación del ciclo de actividad (duty cycle) para regular la potencia o intensidad entregada a un dispositivo?
a. Lectura analógica de un sensor de temperatura.
b. Generación de interrupciones de tiempo real del sistema.
c. Control de luminiscencia de un LED.
d. Comunicación serie asíncrona entre microcontroladores.

38. ¿Cómo se define formalmente el ciclo de actividad (duty cycle) de una señal PWM periódica?
a. El porcentaje del tiempo total del período en el que la señal permanece en estado alto.
b. La cantidad de repeticiones completas del pulso que ocurren en un segundo.
c. El tiempo en microsegundos que tarda el microcontrolador en ejecutar la rutina de interrupción.
d. El nivel de tensión pico que alcanza el pulso en el pin del microcontrolador.

39. Al configurar un canal de temporizador en modo de comparación de salida `TIM_OCMODE_PWM1` en modo de conteo ascendente con polaridad alta (`TIM_OCPOLARITY_HIGH`), ¿cuál es el comportamiento de la señal de salida?
a. Mantener la salida en nivel alto constante durante todo el ciclo sin importar el valor de la autorecarga.
b. Permanecer activa (nivel alto) mientras el contador (CNT) sea menor que el registro CCRx, y pasar a inactiva el resto del período.
c. Alternar el nivel lógico del pin en cada desbordamiento del contador sin importar el valor de CCRx.
d. Permanecer inactiva (nivel bajo) mientras el contador (CNT) sea menor que el registro CCRx, y pasar a activa al coincidir.

40. En la configuración de los registros de un temporizador en STM32, ¿qué función cumple el prescalador (`PSC`) para la generación de una señal PWM?
a. Dividir la frecuencia del reloj de entrada del temporizador para ajustar la frecuencia de incremento del contador (CNT).
b. Fijar el tiempo de retardo entre la llegada del pulso y la activación del canal en el pin GPIO.
c. Limitar el número máximo de pulsos emitidos antes de apagar automáticamente el canal.
d. Invertir la polaridad lógica del pin de salida cuando el temporizador entra en estado inactivo.

41. En una comparación entre los protocolos I2C y SPI, ¿cuál es una ventaja clave de la arquitectura I2C con respecto al uso de pines cuando se agregan múltiples dispositivos secundarios al bus?
a. Permite conectar múltiples nodos utilizando únicamente dos líneas compartidas (SDA y SCL) sin añadir líneas físicas adicionales de selección por cada dispositivo.
b. Incrementa la velocidad de transmisión hasta 100 MB/s superando ampliamente los límites de SPI sin importar la capacitancia del bus.
c. Admite comunicación Full-Duplex que transmite datos simultáneamente por líneas dedicadas de entrada y salida sin necesidad de líneas de reloj.
d. Elimina la necesidad de utilizar resistencias de pull-up externas al emplear un esquema de transmisión bidireccional mediante cuatro cables.

42. En un sistema I2C con múltiples controladores principales (multi-main), ¿cómo se resuelve la colisión si dos principales comienzan a transmitir datos al mismo tiempo?
a. El principal que trabaja a una frecuencia de reloj SCL más alta toma el control exclusivo eliminando los datos del otro principal.
b. El principal que intenta enviar un nivel alto pero detecta que SDA está en nivel bajo pierde la arbitración y abandona su transmisión inmediatamente.
c. El bus se reinicia automáticamente mediante una interrupción de tiempo de espera (timeout) de 35 ms enviada a todos los nodos.

+3 -2

d. El dispositivo secundario envía un bit NACK a ambos principales para forzarlos a reintentar la transmisión tras un retraso aleatorio.

43. ¿Cuál es la diferencia principal en la gestión de líneas de selección de chip (CS) entre una configuración SPI regular con múltiples secundarios y una en cadena de margarita (daisy-chain)?

a. En la configuración regular cada secundario requiere una línea CS dedicada, mientras que en cadena de margarita todos los secundarios comparten una única línea CS conectada en paralelo.
b. La configuración en cadena de margarita elimina por completo las líneas MOSI y MISO sustituyéndolas por pines CS bidireccionales.
c. La cadena de margarita requiere que el principal asigne una dirección binaria de 7 bits a cada secundario a través del pin CS.
d. En la configuración regular los datos se desplazan bit a bit a través de cada secundario, requiriendo más ciclos de reloj por dispositivo.

44. ¿Qué función cumple la señal opcional SMBALERT# en la arquitectura del bus SMBus?

a. Sustituye a la línea de reloj SMBCLK durante las transferencias de datos en modo rápido.
b. Fuerza la reconexión de las resistencias de pull-up principales cuando la capacitancia del bus excede los 400 pF.
c. Permite que un dispositivo secundario notifique de manera rápida e interrumpa al principal cuando ocurre un evento o condición de falla.
d. Proporciona la fuente de alimentación principal de 3.3 V a todos los nodos conectados en el bus.

45. ¿Cuál es la restricción clave que deben cumplir los nombres de los Workspaces, proyectos y sus respectivas rutas (paths) en STM32CubeIDE?

a. Deben contener exclusivamente caracteres ASCII.
b. Deben incluir obligatoriamente el sufijo de la placa de desarrollo (ej. _F103RB).
c. Deben configurarse como rutas absolutas en formato POSIX sin importar el sistema operativo.
d. Deben estar alojados siempre en la raíz del disco principal C:\.

46. Para poder visualizar la salida de comandos como `printf()` por la consola de depuración cuando se utiliza OpenOCD, ¿qué comando de inicialización debe ejecutarse en la pestaña *Startup* de la configuración de depuración?

a. openocd -f stlink.cfg
b. monitor arm semihosting enable
c. semihosting on --verbose
d. target remote localhost:3333

47. ¿Qué representa la métrica WCET contenida en las estructuras de datos de las tareas en el proyecto del ejecutor cíclico?

a. La suma total acumulada de microsegundos que la tarea ha estado en estado bloqueado.
b. El tiempo de ciclo total de trabajo del procesador (Work-Cycle Total Execution).
c. El tiempo de ejecución en el peor de los casos (Worst-Case Execution Time) medido en microsegundos.
d. El tiempo de espera de respuesta del canal de comunicación (Waiting Communication Extended Time).

48. ¿Qué función debe agregarse dentro de la rutina de interrupción `SysTick_IRQn` en el archivo `stm32f1xx_it.c` para permitir la correcta actualización del contador de ticks global de la capa HAL?

a. HAL_SYSTICK_IRQHandler();
b. SysTick_Config();
c. app_update();
d. HAL_IncTick();

49. ¿Cuál es la división tripartita de módulos de código C de tipo temporizado (período de 1 ms) propuesta en la arquitectura del proyecto para separar adecuadamente las responsabilidades de software?

a. Módulo Sensor para escrutar, módulo System para procesar y módulo Actuator para actuar.

+1 -1

b. Módulo Main para temporizar, módulo Logic para evaluar estados y módulo Display para la interfaz visual.
c. Módulo Input para leer interrupciones, módulo Control para la base de datos y módulo Output para drivers.
d. Módulo Hardware, módulo Firmware y módulo Software.

50. ¿Qué fenómeno mecánico se presenta al accionar un pulsador y de qué recurso se vale la máquina de estados en el modelo Sensor para mitigar sus efectos?

a. El rebote del contacto ("switch bounce"), el cual se filtra mediante el uso de temporizadores (timers/tick) para estabilizar la señal.
b. El bloqueo por histéresis analógica, que se corrige sustituyendo todos los pulsadores por dip switches.
c. El ruido electromagnético de alta frecuencia, que se elimina deshabilitando las interrupciones globales.
d. La caída de tensión del bus de datos, la cual se compensa aumentando la frecuencia del reloj interno.

51. En el modelo temporal de un pulsador presentado en la guía, ¿cuál es la secuencia correcta de estados por la que pasa la señal al transicionar desde que el botón está liberado hasta que queda presionado de forma estable?

a. BUTTON_UP → BUTTON_FALLING → BUTTON_DOWN
b. BUTTON_DOWN → BUTTON_FALLING → BUTTON_UP
c. BUTTON_UP → BUTTON_RISING → BUTTON_DOWN
d. BUTTON_FALLING → BUTTON_UP → BUTTON_RISING

52. ¿Qué extensión de archivo poseen los modelos de diagramas de estado creados y guardados localmente mediante la versión de escritorio de Itemis Create?

a. .txt
b. .xlsx
c. .syc
d. .scm

53. En la arquitectura de ejecutor cíclico no bloqueante basada en temporización por software, ¿cuál es la función principal del método `HAL_SYSTICK_Callback()`?

a. Calcular el tiempo en el peor de los casos (`WCET`) y reiniciar la pila de memoria RAM.
b. Actualizar el contador general de ticks de la aplicación y de aquellas tareas temporizadas que requieran sincronización.
c. Reconfigurar la frecuencia del reloj del sistema `SystemCoreClock` en cada intervalo de tiempo.
d. Ejecutar directamente la lógica de estado de cada tarea (`task_xxxx_update()`) dentro del contexto de la interrupción.

54. En el contexto de un ejecutor cíclico no bloqueante, ¿por qué está contraindicado el uso de funciones que contengan código bloqueante (como llamadas a retardo por software dentro de `task_xxxx_update()`)?

a. Porque detiene la ejecución del hilo principal, impidiendo que el ejecutor avance hacia las demás tareas y degradando el determinismo del sistema.
b. Porque inhabilita inmediatamente las interrupciones del temporizador SysTick a nivel de hardware.
c. Porque genera una falla de compilación automática al detectar palabras clave no permitidas.
d. Porque duplica el consumo de memoria Flash asignada al vector de interrupciones.

55. En un actuador tipo LED configurado en modo 'Active High' conectado a una salida GPIO y referenciado a masa (GND) mediante su resistencia limitadora, ¿cómo opera la salida del microcontrolador para encender el LED?

a. El pin se establece en nivel alto (1 lógico / VCC), entregando corriente (*source*) desde el microcontrolador hacia el LED.
b. El pin se establece en nivel bajo (0 lógico / GND), absorbiendo corriente (*sink*) desde la fuente del LED.

c. El pin se configura temporalmente como entrada analógica para elevar la corriente de conducción.
d. El pin conmuta a alta frecuencia utilizando una interrupción de temporizador de captura/comparación.

56. En el archivo `main.c`, ¿cuál es la función de llamar a `HAL_Init()` y `SystemClock_Config()` antes de invocar `app_init()`?

a. Inicializar el tiempo de la capa de abstracción de hardware (HAL), configurar el reloj del sistema `SystemCoreClock` y habilitar el temporizador base SysTick.
b. Ejecutar inmediatamente la primera máquina de estados de los actuadores para comprobar el encendido de los LEDs.
c. Crear el repositorio Git e inicializar el semihosting en la consola de depuración de STM32CubeIDE.
d. Limpiar la memoria RAM y comprobar que los archivos de documentación `.txt` existan en la tarjeta SD.

57. Al portar la gestión de hardware desde la librería Mbed C++ a STM32 HAL en lenguaje C, ¿cuál es la función adecuada para reemplazar la escritura directa en un pin de salida digital como `displayD4 = value;`?

a. HAL_GPIO_WritePin((GPIO_TypeDef *)D4_GPIO_Port, (uint16_t)D4_Pin, (GPIO_PinState)value);
b. HAL_GPIO_ReadPin((GPIO_TypeDef *)D4_GPIO_Port, (uint16_t)D4_Pin);
c. displayD4.write((uint8_t)value);
d. HAL_GPIO_TogglePin((GPIO_TypeDef *)D4_GPIO_Port, (uint16_t)D4_Pin);

58. En el circuito de interfaz para una pantalla LCD alfanumérica, ¿cuál es la función del potenciómetro de 10 kΩ conectado entre 5V, GND y el pin VO del display?

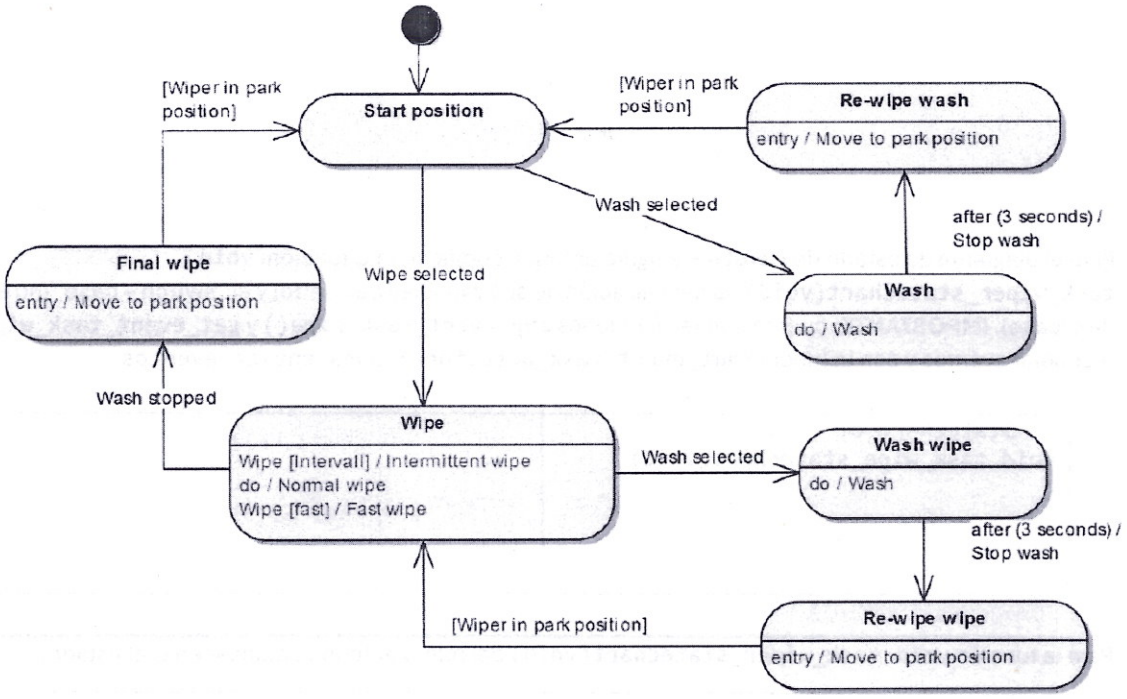
a. Seleccionar entre el modo de lectura y el modo de escritura de los registros del controlador HD44780.
b. Ajustar la velocidad de transferencia del reloj durante la transmisión de comandos por los pines D4-D7.
c. Regular la tensión analógica aplicada al pin VO para ajustar el contraste de los caracteres presentados.
d. Limitar la corriente enviada al anodo del LED de retroiluminación para evitar daños al display.

59. En un menú interactivo de múltiples niveles controlado por tres botones (*Enter*, *Next* y *Escape*), ¿qué acción realiza habitualmente el evento asociado al botón *Next*?

a. Navegar o alternar horizontalmente entre las distintas opciones disponibles dentro del mismo nivel de menú.
b. Cancelar la selección actual y regresar inmediatamente al menú principal (*Main*).
c. Descender un nivel en la jerarquía del menú para ingresar a la configuración del submenú seleccionado.
d. Incrementar en un segundo el contador de tiempo de ejecución de la tarea del sistema.

60. En la aplicación con menú interactivo y teclado de membrana, ¿cuál es el efecto de presionar el botón *Enter* (conectado a D10) sobre la línea 2 del display LCD al mostrar el mensaje 'Test N°: XXXX'?

a. Reiniciar permanentemente el contador XXXX a cero y borrar la línea de bienvenida del LCD.
b. Modificar la frecuencia del oscilador del microcontrolador para acelerar la actualización del LCD.
c. Conmutar el estado para alternar entre presentar o no presentar en pantalla el contador incrementado cada segundo.
d. Cambiar dinámicamente la comunicación del LCD desde el modo de 4 bits hacia el modo de 8 bits.



State diagram for a wiper system

61. **Para** el diagrama de estado de **Harel** de la figura anterior (**Wiper system**), declarar **enumeraciones** (**eventos** y **estados**) y **estructuras de datos**.

```
/* Events to excite Task Wiper */
typedef enum task_wiper_ev { xxx , yyy, zzz } task_wiper_ev_t;
/* States of Task Wiper */
typedef enum task_wiper_st { xxx , yyy, zzz } task_wiper_st_t;
/* Events to excite Task Actuator */
typedef enum task_actuator_ev { xxx , yyy, zzz } task_actuator_ev_t;

typedef struct
{
      xxx_t        xxx;
      yyy_t        yyy;
      zzz_t        zzz;
} task_wiper_dta_t;
```

---

62. Para el diagrama de estado de **Harel** de la figura anterior, codificar en C función: `void task_wiper_init(void)`, `void task_wiper_update(void)`.

```
/* Init of Task Wiper */
void task_wipe_init(void)
{
      . . .
}

/* Update of Task Wiper */
void task_wipe_update(void)
{
      . . .
}
```

---

63. Para el diagrama de estado de **Harel** de la figura anterior, codificar en C función: `void` **`task_wiper_statechart`**`(void)`, la inicialización de sus variables de control y el **"switch – case"** (no el contenido de c/**case**). **IMPORTANTE**: cuenta con las funciones **`any_event_task_wipe()`** y **`get_event_task_wipe()`** para recuperar eventos y con la función **`put_event_task_actuator()`** `para enviar eventos.`

```
/* Statechart of Task Wiper */
void task_wipe_statechart(void)
{
      . . .
}
```

---

64. Para la función `void` **`task_wiper_statechart`**`(void)` del punto anterior, codificar en C el estado:

```
      case Start_position:

            . . .

            break;

      case Wash:

            . . .

            break;

      case Re-wipe_wash:

            . . .

            break;
```

---

65. Para la función `void` **`task_wiper_statechart`**`(void)` del punto anterior, codificar en C el estado:

```
      case Start_position:

            . . .

            break;

      case Wipe:

            . . .

            break;

      case Final_wipe:

            . . .

            break;
```

(61) typedef enum task_wiper_ev { ~~start position~~ wash selected, wipe selected, wash stopped } task_wiper_ev_t;

wipe, idle

typedef enum task_wiper_st { star position, wash, wipe, wash wipe, final wipe, Re-wipe wipe, Re-wipe wash } task_wiper_st_t;

typedef enum task_actuator_ev { entry, do, wipe } task_actuator_ev_t;

typedef struct { task_wiper_ev_t wiper_ev; task_wiper_st_t wiper_st; task_actuator_ev_t actuator_ev; } task_wiper_dta_t;

+2